



Materia: “Estructura de Datos”

Campus: Ferrería

Profesor(a): Ricardo Aguilar Garay

Alumno: Abraham R. Bautista Triana

Trabajo: Avance de Proyecto

Proyecto Integrador — Avance

Compresor de Texto con Codificación de Huffman

Estructuras de Datos y Algoritmos — Tecmilenio

A1. Tema y descripción

Este proyecto es un compresor y descompresor de archivos de texto plano (.txt) basado en el algoritmo de codificación de Huffman. La aplicación toma un texto —escrito directamente en la interfaz o cargado desde un archivo— y genera una versión comprimida del mismo asignando códigos binarios más cortos a los caracteres que aparecen con mayor frecuencia y códigos más largos a los caracteres poco frecuentes, en lugar de usar el mismo número de bits para todos los caracteres como ocurre en una codificación estándar. El proceso es completamente reversible: la aplicación también permite descomprimir el resultado y recuperar el texto original de forma exacta, sin pérdida de información.

Esta herramienta sería útil para cualquier persona que quiera entender de forma práctica cómo funcionan los algoritmos de compresión sin pérdida que se usan a diario en formatos como ZIP o PDF, así como para cualquier caso en el que se necesite reducir el espacio ocupado por archivos de texto (documentos, registros, logs) antes de almacenarlos o transmitirlos. A diferencia de un ejemplo puramente teórico, la aplicación permite cargar un archivo real, ver el árbol de decisiones que se construyó a partir de su contenido y comprobar en pantalla el porcentaje real de espacio ahorrado.

A2. Estructuras seleccionadas y justificación

1. Cola de prioridad (min-heap implementado desde cero)

El algoritmo de Huffman construye su árbol de forma voraz: en cada paso debe tomar los dos elementos de menor frecuencia disponibles —ya sean caracteres originales o subárboles ya combinados— y fusionarlos en uno nuevo. Esta operación se repite hasta que solo queda un elemento. Una lista recorrida linealmente para "buscar el mínimo" funcionaría, pero sería ineficiente y, sobre todo, no refleja la naturaleza del problema: lo que se necesita no es una lista ordenada una sola vez, sino una estructura que permita insertar elementos nuevos (los nodos combinados) y extraer el mínimo repetidamente, de forma intercalada, hasta el final del proceso. Eso es exactamente lo que resuelve una cola de prioridad, razón por la cual es la estructura natural para este problema y no una simple ordenación estática.

2. Árbol binario (árbol de Huffman, implementado desde cero)

El árbol de Huffman no es un detalle de implementación: es el resultado central del algoritmo y la única forma correcta de generar códigos binarios que sean "libres de prefijos" (que ningún código sea el inicio de otro), lo cual es indispensable para poder decodificar un mensaje comprimido sin ambigüedad. Cada carácter del texto corresponde a una hoja del árbol, y su código se obtiene leyendo el camino desde la raíz hasta esa hoja. Sin este árbol no habría manera de generar los códigos de compresión ni de descomprimir el mensaje después: es la estructura que sostiene tanto la compresión como la descompresión, por lo que cumple de forma directa con que la estructura sea central y no un adorno.

Ambas estructuras se implementarán con clases propias (NodoHuffman como bloque de construcción, sin usar `java.util.PriorityQueue`, `java.util.TreeMap` ni ninguna clase de estructuras de datos incluida en Java), cumpliendo con la prohibición estricta indicada por el profesor.

A3. Diseño preliminar de clases

A continuación se listan las clases principales previstas para el proyecto, sus atributos más relevantes y su responsabilidad dentro del sistema. El diseño busca separar claramente la lógica de las estructuras de datos (NodoHuffman, ColaPrioridad, ArbolHuffman) de la lógica de la interfaz gráfica (VentanaPrincipal), de modo que cada clase tenga una única responsabilidad bien definida.

Clase	Atributos principales	Responsabilidad
NodoHuffman	caracter, frecuencia, izquierdo, derecho	Representar un nodo del árbol de Huffman, ya sea una hoja (un carácter) o un nodo interno (unión de dos subárboles).
ColaPrioridad	arreglo interno de NodoHuffman, tamaño actual	Implementar un min-heap propio con <code>insertar()</code> y <code>extraerMinimo()</code> , garantizando acceso rápido al nodo de menor frecuencia en todo momento.
AnalizadorFrecuencias	tabla de frecuencias (arreglo indexado por carácter)	Recorrer el texto de entrada y contar cuántas veces aparece cada carácter, como paso previo a construir el árbol.
ArbolHuffman	raíz (NodoHuffman), tablaCodigos	Construir el árbol a partir de las frecuencias usando la ColaPrioridad, generar la tabla de códigos binarios y permitir decodificar bit a bit.
CompresorHuffman	(orquesta AnalizadorFrecuencias y ArbolHuffman)	Coordinar el proceso completo: <code>comprimir(texto)</code> y <code>descomprimir(bits, arbol)</code> , sin exponer los detalles internos de las estructuras a la interfaz.

Clase	Atributos principales	Responsabilidad
VentanaPrincipal	componentes Swing: JTextArea, JButton, JLabel	Mostrar la interfaz gráfica, capturar los eventos del usuario (cargar archivo, comprimir, descomprimir) y delegar la lógica a CompresorHuffman.

A4. Boceto de la interfaz

La aplicación tendrá una sola ventana principal (JFrame) con los siguientes elementos: un botón para cargar un archivo .txt (o un área de texto para escribir directamente), un área de texto que muestra el contenido original, dos botones de acción ("Comprimir" y "Descomprimir"), un panel de estadísticas que muestra el tamaño original, el tamaño comprimido y el porcentaje de ahorro, y un área de resultado donde se muestra la cadena de bits comprimida o el texto reconstruido, según la operación realizada. A continuación se muestra el mockup de esta pantalla:

Compresor de Texto (Huffman)

Cargar archivo .txt

nombre_archivo.txt (ninguno cargado)

Texto original:

(contenido del texto cargado o escrito aqui)

Comprimir

Descomprimir

Tamano original: 0 bits Tamano comprimido: 0 bits Ahorro: 0 %

Resultado (bits comprimidos / texto descomprimido):

(aqui se muestra la cadena de bits al comprimir,
o el texto reconstruido al descomprimir)

Como trabajo futuro (fuera del alcance del avance), se contempla agregar una segunda vista opcional que dibuje visualmente el árbol de Huffman generado, útil para la demostración en la entrega final.

A5. Estructura ya implementada y probada desde consola

Se implementaron y probaron desde consola las dos clases que forman la primera de las estructuras centrales del proyecto: `NodoHuffman` (el bloque de construcción, tanto para hojas como para nodos internos) y `ColaPrioridad` (el min-heap propio sobre un arreglo nativo, sin usar `java.util.PriorityQueue`). El código fuente completo de ambas clases se entrega junto con este documento, dentro de los archivos `NodoHuffman.java` y `ColaPrioridad.java`.

Para comprobar que la cola de prioridad ordena correctamente por frecuencia, se escribió una clase de prueba (`PruebaColaPrioridad.java`, incluida también en la entrega) que inserta seis nodos con frecuencias desordenadas a propósito (15, 4, 22, 7, 1, 9) y los extrae uno por uno con `extraerMinimo()`, además de comprobar el caso límite de extraer de una cola ya vacía. La salida obtenida en la consola de VS Code al ejecutar esta prueba fue la siguiente:

```
Tamano despues de insertar: 6
Orden de salida (deberia ser ascendente):
1
4
7
9
15
22
Extraer de cola vacia (deberia ser null): null
```

Los seis nodos insertados salieron en orden ascendente (1, 4, 7, 9, 15, 22) sin importar el orden desordenado en que fueron insertados, lo cual confirma que la reorganización interna del heap (los métodos `flotar()` y `hundir()`) funciona correctamente. Asimismo, extraer de una cola ya vacía devolvió `null` en lugar de generar un error, cubriendo ese caso límite desde esta etapa temprana del proyecto.